Ituring /
VDO_SLAM_modified

Code · Issues 3 · Pull requests · Actions · Projects · Security · Insights

动态slam

Unknown, GPL-3.0 licenses found
25 stars · 7 forks · 1 watching · Branches · Activity · Tags

Public repository

1 Branch · 0 Tags · Go to file · t · Go to file · + · Add file · Code · ...

Ituring Update README.md · 55182e6 · last year

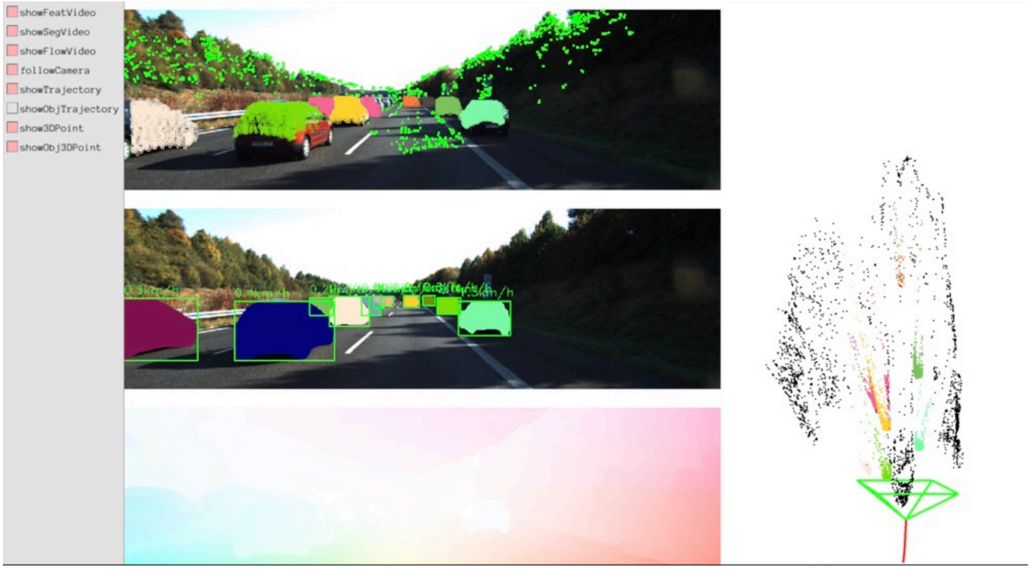
VideoFlow	update readme	last year
YOLOv8	update gitignore and upload files	last year
ZoeDepth	update readme	last year
cmake_modules	upload	last year
dependencies/g2o	upload	last year
example	update readme	last year
include	upload	last year
src	update tracking.cc, any customed ...	last year
.gitignore	update readme	last year
CMakeLists.txt	upload	last year
LICENSE-GPL.txt	upload	last year
LICENSE.txt	upload	last year
README.md	Update README.md	last year
build.sh	upload	last year
how_to_run.txt	upload	last year
onnxruntime-linux-x64-1.16....	upload	last year
sourcecode_guidance.md	upload	last year
vdo_slam.png	update gitignore and upload files	last year

README · License · GPL-3.0 license

在动态环境中，相比剔除动态物体上的特征点，VDO_SLAM结合图像分割、光流估计以及深度估计等算法，除了估算相机位姿外，还估算了动态物体的位姿以及跟踪

b站视频

- vdo_slam



example

准备工作

1. kitti测试数据下载
   链接：kitti_10_03
   相关参数：calib
2. 参照VideoFlow，生成光流估计
3. 参照ZoeDepth，生成深度信息
4. 从yolov8下载yolov8 seg onnx，比如yolov8 seg small，并保存在当前目录下
5. opencv-4.8, pangolin-0.6，eigen-3.4.0，解压onnxruntime-linux-x64-1.16.3.tgz

运行

```
./example/vdo_slam example/kitti_10_03.yaml
/home/spurs/dataset/kitti_raw/2011_10_03/2011_10_03_drive_0047_sync/image_02
```

相关的原理

记${}^0A_k$和${}^0L_k$分别为第k帧到第0帧的相机位姿和物体位姿，一般第0帧所在坐标系为世界坐标系。记${}^0m_k^i$ 为第k帧上的第i个特征点对应在世界坐标系的3d坐标，并且为了方便计算，记

$$ {}^0m_k^i = [m_x^i, m_y^i, m_z^i, 1] $$

则

$$ {}^{X_k}m^i = {}^0X_k^{-1} \cdot {}^0m_k^i $$

记p_k^i为第k帧上第i个特征点的2d位置，为了方便计算，记

$$p_k^i = [u^i, v^i, 1]$$

设K为相机内参，则

$$p_k^i = K^{X_k} m^i$$

记${}^{L_{k-1}}_{k-1}H_k$为物体从第k帧对应时刻到第k-1帧对应时刻在第k-1帧对应坐标系下的相对位姿，则

$${}^{L_{k-1}}_{k-1}H_k = {}^0L_{k-1}^{-1} \cdot {}^0L_k$$

则

$${}^{L_k}m_k^i = {}^0L_k^{-1} \cdot {}^0m_k^i$$

则

$${}^0m_k^i = {}^0L_k \cdot {}^{L_k}m_k^i = {}^0L_{k-1} \cdot {}^{L_{k-1}}_{k-1}H_k \cdot {}^{L_k}m_k^i$$

以及对于刚体有

$${}^{L_k}m_k^i = {}^{L_{k-1}}m_{k-1}^i$$

则

$${}^0m_k^i = {}^0L_{k-1} \cdot {}^{L_{k-1}}_{k-1}H_k \cdot {}^{L_{k-1}}m_{k-1}^i = {}^0L_{k-1} \cdot {}^{L_{k-1}}_{k-1}H_k \cdot {}^0L_{k-1}^{-1} \cdot {}^0m_{k-1}^i$$

记

$${}^0_{k-1}H_k = {}^0L_{k-1} \cdot {}^{L_{k-1}}_{k-1}H_k \cdot {}^0L_{k-1}^{-1}$$

则

$${}^0m_k^i = {}^0_{k-1}H_k \cdot {}^0m_{k-1}^i$$

则根据相邻帧上的${}^0m$求刚体的速度，即

$$v_k = \frac{{}^0m_k - {}^0m_{k-1}}{\delta t} = \frac{({}^0_{k-1}H_k - I) \cdot {}^0m_{k-1}}{\delta t}$$

其中δt 为相邻帧对应的时间间隔，记

$${}^0_{k-1}H_k = \begin{bmatrix} {}^0_{k-1}R_k & {}^0_{k-1}t_k \\ 0 & 1 \end{bmatrix}$$

则

$$v_k = \frac{{}^0_{k-1}t_k - (I_3 - {}^0_{k-1}R_k) \cdot {}^0m_{k-1}}{\delta t}$$

相关的论文

- [Robust Ego and Object 6-DoF Motion Estimation and Tracking]
- [VDO-SLAM: A Visual Dynamic Object-aware SLAM System]

▶ **offical readme**(click to expand)

Releases

No releases published

Packages

No packages published

Languages

- Python 40.3%
- C++ 31.0%
- Jupyter Notebook 18.1%
- Java 5.9%
- Swift 3.1%
- CMake 0.9%
- Other 0.7%